IST 387: Introduction to Applied Data Science
Lab 6: Notes

Notes Prepared by: Dr. Preeti Jagadev

Reference: Data Science for Business with R, written by Jeffrey S. Saltz and Jeffrey M. Stanton

Contents of the Notes:

- A Visualization Overview
- Basic Plots in R
- Using the GGPLOT2 Package
- More Advanced Visualizations
- Important R Functions

A Visualization Overview

Visualization is the communication of information using graphical representations.

Information Visualization is the use of visual representations of abstract data. Information visualization is used when there is no well-defined two dimensional or three-dimensional representation of the data.

There are six key components one can use to create a visualization. To help think about these components, let's imagine a simple scatterplot with dots (or other symbols) arrayed on a far surface (like a computer screen) with an X-axis along the bottom and a Y-axis along the side.

Here are the six components:

1. **Proximity/location**: The position of the symbol relative to the X- and Y-axes.

2. **Color:** The color of each symbol. Note that 7-10% of men have red-green color blindness, so the use of color needs to take this into account.

3. **Size:** The size of each symbol in the scatterplot.

4. **Texture:** The shape of the symbol and whether the symbol is a solid color or a pattern.

5. **Annotation:** Labels and legends that we use on our scatterplots.

6. **Interactivity:** The capability to select one or more symbols and get information about it, as well as zooming, animation, and other features.

A common mistake is to create a visualization that contains a lot of information but that is difficult to understand. To address this concern, we describe 10 principles that should be useful to think about as you create a visualization.

1) **Simplicity:** You should create the simplest graph that conveys the information you want to convey.

2) **Encoding:** Consider the type of encoding used to connect a numeric value to a graphical feature and try to make the encoding intuitive. For example, a larger numeric value should be represented by a larger dot and a smaller value by a smaller dot.
3) **Patterns versus details:** Make a choice whether to highlight big picture patterns or fine-level details. It is very hard to do both in the same graphic.

4) **Ranges:** Select meaningful ranges for the axes.

5) **Transformations**: Transforming data can be useful, particularly when a set of numeric values cover a very large range. For example, a natural log transformation can compress the range 1-1,000 into a seven-point scale.

6) **Density:** Use transparency to represent data where many points are plotted near to or on top of one another. Another useful strategy to avoid the problem of "overplotting" is to use jitter —a technique that moves points very slightly and randomly.

7) **Connections:** Use lines to connect sequential data. This is particularly important when showing a time series.

8) **Aggregates:** Combine data by creating meaningful groups.

9) **Comparison:** Keep axis ranges as similar as possible when comparing multiple graphs.

10) **Color:** Select an appropriate color scheme, based on the meaning of the data. For example, gradations of heat might be represented with shades of blue and red, whereas the density of plant life might be shown with yellow and green.

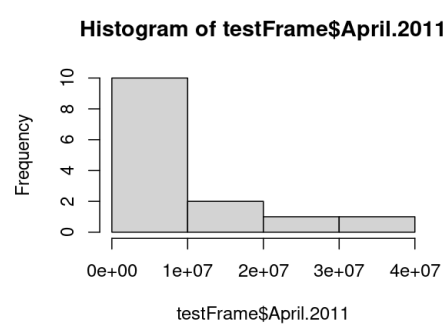
Basic Plots in R

Now, let's create a R code to read this dataset (**example2.csv**):

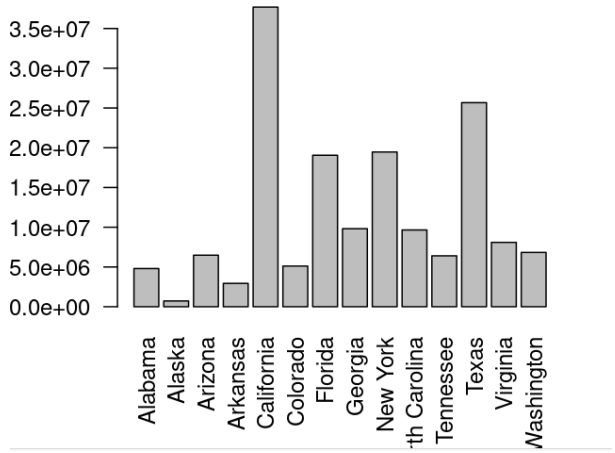
Region	Census	Estimate	April.2010	April.2011
Ala.bama	4,779,736	4,779,735	4,785,401	4802740
Alaska.	710,231	710,231	714,146	722718
.Arizona	6,392,017	6,392,013	6,413,158	6482505
Arkansas.	2,915,918	2,915,921	2,921,588	2937979
California.	37,253,956	37,253,956	37,338,198	37691912
Colo.rado	5,029,196	5,029,196	5,047,692	5116796
Fl.orida.	18,801,310	18,801,311	18,838,613	19057542
Georgia.	9,687,653	9,687,660	9,712,157	9815210
New York.	19,378,102	19,378,104	19,395,206	19465197
North Carolina.	9,535,483	9,535,475	9,560,234	9656401
.Tennessee	6,346,105	6,346,110	6,357,436	6403353
Tex.as	25,145,561	25,145,561	25,253,466	25674781
Virginia.	8,001,024	8,001,030	8,023,953	8096604
Washington.	6,724,540	6,724,540	6,742,950	6830038

```
install.packages("readr")
library (readr)
testFrame <- read_csv("example2.csv")
testFrame$Region <- gsub("\\.", "", as.character(testFrame$Region))
testFrame$change <- testFrame$April.2011 - testFrame$April.2010
hist(testFrame$April.2011)
```

The output is as follows:



Using R base graphics, we can show a bar plot:



Using the GGPLOT2 Package

Although doing these simple charts in R's base graphics is very easy, if we want to create more advanced visualizations, we need to explore an advanced visualization package, known as ggplot2, which is part of the tidyverse set of packages.

ggplot2 is a plotting system for R, which tries to keep the good parts of base and Lattice graphics and discard the bad parts. It takes care of many of the fiddly details that make porting a hassle (like drawing legends) as well as providing a powerful model of graphics that makes it easy to produce complex multi-layered graphics. The "gg" stands for grammar of graphics. The package is now the most used visualization tool within the R ecosystem.

The concept behind ggplot2 is that it divides the definition of a plot into three different fundamental parts: **Plot = data + Aesthetics + Geometry**. The components of every plot can be defined as follows:

- **Data:** Needs to be a data frame.
- **Aesthetics:** Is used to indicate x and y variables. It can also be used to control the color, the size, or the shape of points, the height of bars, and so on.
- **Geometry:** Defines the type of graphics (histogram, box plot, line plot, density plot, dot plot, ...).

Before we get started using ggplot, we need to ensure that ggplot2 has been installed and libraried. Rather than doing this manually, let's create a function that will assist us and make the activity more repeatable. The function will take as input the name of a package. It then tests whether the package has been downloaded-installed-from the R code repository. If it has not yet been downloaded/installed, the function takes care of this. Then we use a new function, called **require()** to prepare the package for further use.

Let's call our function **EnsurePackage**, because it ensures that a package is ready for us to use:

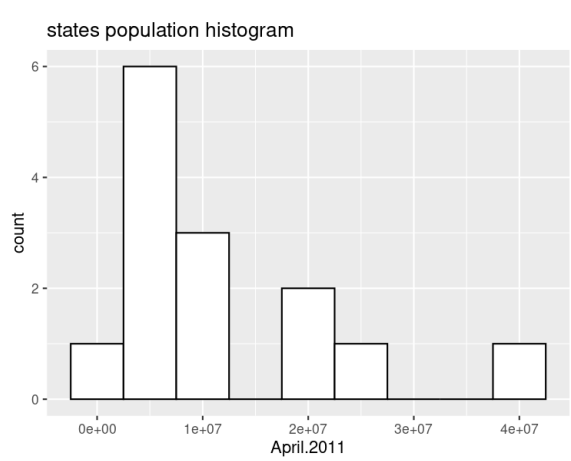
```
EnsurePackage <- function(x) {
  x <- as.character(x)

  if (!require(x, character.only = TRUE)) {
    install.packages(pkgs=x, repos="http://cran.r-project.org")
    require(x, character.only = TRUE)
  }

}
```

The **require()** function does the same thing as **library( )**, but it also returns the value FALSE if the package you requested in the argument x has not yet been downloaded. That same line of code also contains another new feature, the "if" statement. It rests the stuff inside the parentheses to see if it evaluates to TRUE or FALSE. If TRUE, the program continues to run the script in between the inner pair of curly braces. If FALSE, all the stuff in the curly braces is skipped. Also, in the third line, in case you are curious, the arguments to the **require ()** function include x, which is the name of the package that was passed into the function, and character.only=TRUE, which tells the **require( )** function to expect x to be a character string. Last thing to notice about this third line: there is an exclamation mark (!) character that reverses the results of the logical test. Technically, it is the Boolean function NOT. It requires some mental gyration that when require returns FALSE, the ! inverts it to TRUE, and that is when the code inside the inner curly braces runs.

Now we are ready to put **EnsurePackage** to work:

```
EnsurePackage("tidyverse")
#The data
g <- ggplot(testFrame)

#Add the aesthetic to the plot variable
g<- g + aes(x=April.2011)

#add the geometry
g <- g + geom_histogram(binwidth = 5000000, color="black", fill="white")

#add the title
g <- g + ggtitle("states population histogram")
g
```

The output is as follows:



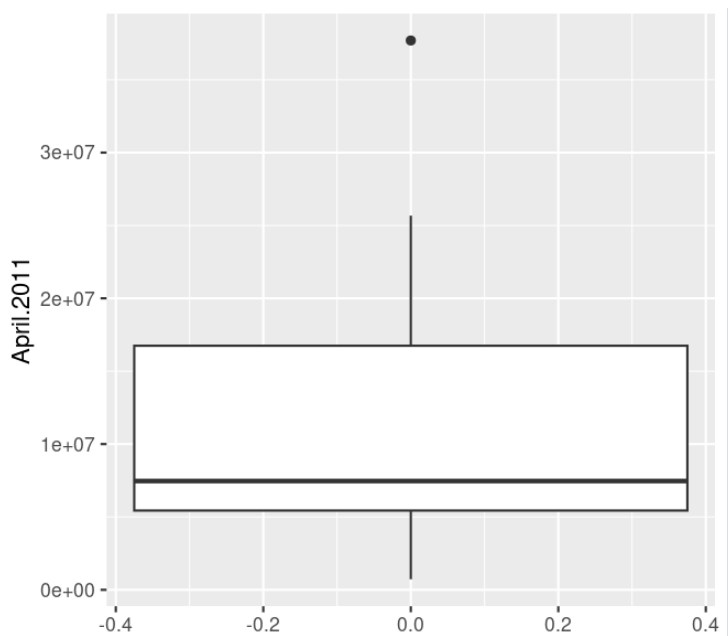
To review the code, the first line creates a ggplot, where the data frame to be used is testFrame. Note that once we specify the data frame, ggplot looks at the columns within the database (such as April.2011).

The next line of code defines the aesthetics, specifically, that the X-axis will be the April.2011 population. On the following line, the geometry is defined, by stating that we want to use a histogram with the bars being white with a black outline; finally, we add a title.

Another way to examine the distribution of a set of numbers is to create a boxplot. A boxplot consists of a box, whiskers, and outliers. The box contains the middle 50% of the data, whereas the whiskers cover about 99% of the data, not counting outliers. The boxplot procedure has an algorithm to decide on what data points get shown as an outlier-often this contains any point that is farther away from the box than 1.5 times the width of the box.

Note that the code below takes a different strategy from the previous code example by putting the data, aesthetic, and geometry all on one line. We also did nor store the resulting visualization in a variable. For simple graphics that render quickly, there's no need to store the ggplot specification in a variable because we can simply run the complete line of code again if we want to regenerate the graphic. Here's the code and the resulting plot:

```
ggplot(testFrame) + aes(y=April.2011) + geom_boxplot()
```

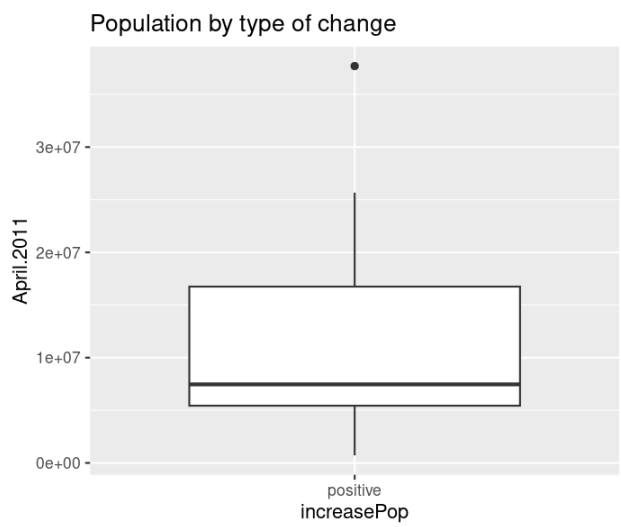


We can explore the distribution of the population a bit more in depth by putting the states into two groups-one group for those states with an increase in population and one group for states with a decrease in population. We can use the following code, where the first line creates a new column in the data frame, based on the change in population. The second line of the code creates a new column in the data frame, noting if the population change was positive or negative. We use this as

a grouping variable: states that grew go into the positive group and states that shrank go in the negative group:

```
# create the two new attributes
testFrame$popChange <- testFrame$April.2011 - testFrame$April.2010

testFrame$increasePop <- ifelse(testFrame$popChange > 0,
                                "positive", "negative")

# now generate / display the boxplot
ggplot(testFrame) +
  aes(x=increasePop, y=April.2011) +
  geom_boxplot() +
  ggtitle('Population by type of change')
```

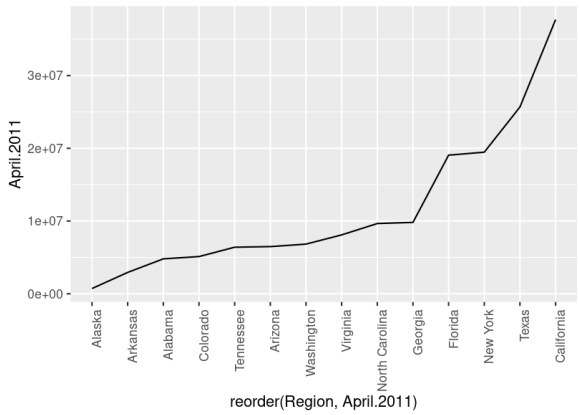
The output is as follows:

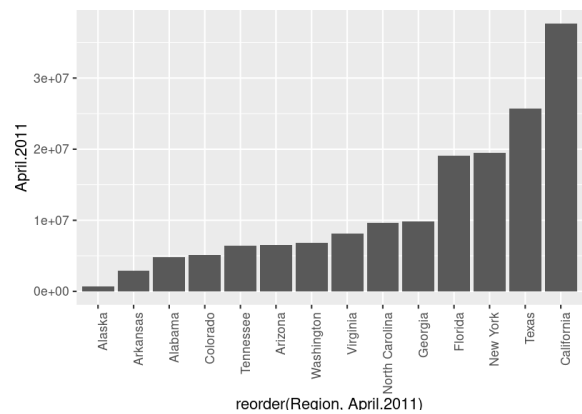


In addition to histograms and boxplots, we can also use ggplot2 to create line and bar charts. To create a line chart, we use the **geom_line( )** function, to create a bar chart, we use the **geom_col( )** function. You can see in the code below that we rotate the x labels (the state names) so that we can easily read the state name. Finally, note that the height of the bar (or line) represents the value in a column of our data frame.

```
ggplot(testFrame) +
  aes(x=reorder(Region, April.2011),
      y=April.2011, group=1) +
  geom_line() +
  theme(axis.text.x = element_text(angle = 90,
                                   hjust = 1))


ggplot(testFrame) +
  aes(x=reorder(Region, April.2011),
      y=April.2011) +
  geom_col() +
  theme(axis.text.x = element_text(angle = 90,
        hjust = 1))
```

The output is as follows:



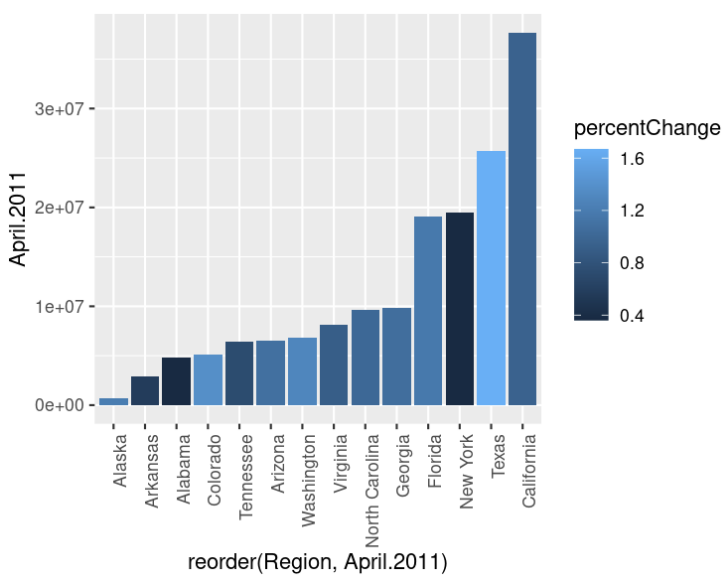


More Advanced Visualizations

We can refine the bar chart by having the color of the bars represent another variable. So, lets create a visualization where the bar color represents the percent change in population:

```
#visualize the % change attribute
testFrame$percentChange <-
  testFrame$popChange/testFrame$April.2010 * 100

#make the % change
ggplot(testFrame) +
  aes(x=reorder(Region, April.2011),
      y=April.2011, group=1, fill=percentChange) +
  geom_col() +
  theme(axis.text.x = element_text(angle = 90,
                                   hjust = 1))
```

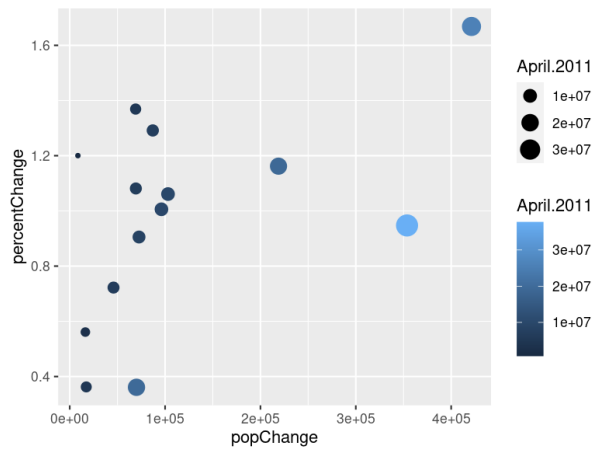
The output is as follows:



The last type of visualization we will explore is a scatterplot, which provides a way to look at the data across two dimensions. Let's create a scatterplot where each point represents a state. We can place the point on the two-dimensional grid based on the population change (x-axis) and the percent change (y-axis). In addition, we color the points based on the April 2011 population. Just to demonstrate the capability, we use that population for the size of the point as well.

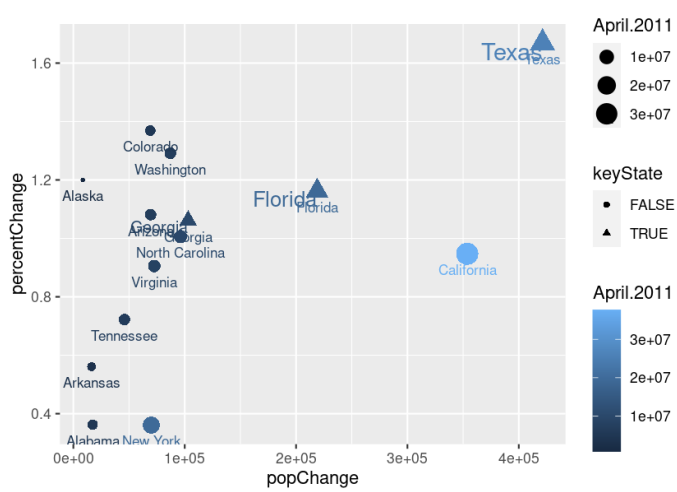
```
g <- ggplot(testFrame) +
  aes(x=popChange, y=percentChange,
      size=April.2011, color=April.2011) +
  geom_point()
g
```

The output is as follows:



You can see we saved the visualization in a variable ("g"), so we can add another layer. Let's use that stored visualization variable and add some labeling to show the name of each state. Note that we defined some additional aesthetics for the text (the size, and the vertical adjustment of where to place the text).

```
g + geom_text(size=3, vjust=2) + aes(label=Region)
```



This chart has now become very cluttered. Let's clean it up by doing several actions:

- First, let's define a set of key (or important) states. These are the states that have a percentage change of at least 1% and a population change of at least 100,000 people. With these criteria, we define a new column in the **testFrame** data frame, **keyState,** to be true if that state fits our defined criteria of percentage and population change.

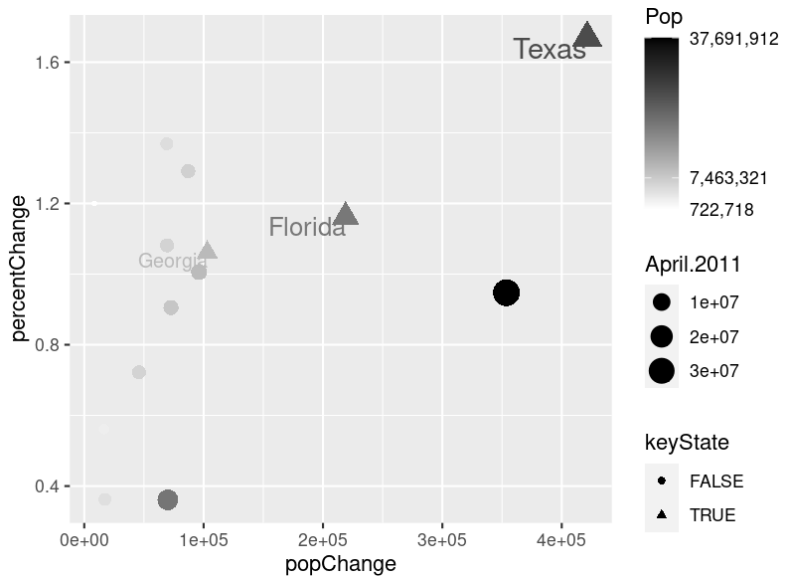
- In the scatter chart, we show the key states by defining the shape of the symbol within the scatterplot to depend on the **keyState** column. We also only show the text for the key states.

- Next, we clean up the format of the color key, defining the three values to be shown in the key as well as formatting the numbers to include commas, so the numbers are easier to see.

- Finally, we change the color scale to range from white to black.

```
#define the criteria to show the text for the state

minPerChange <- 1
minPopChange <- 100000

#determine the key states
testFrame$keyState <- testFrame$popChange > minPopChange &
  testFrame$percentChange > minPerChange

#create a new dataframe of just the key states
keyStateDF <- testFrame[testFrame$keyState,]

#define labels showing min, max and median
minLabel <- format(min(testFrame$April.2011),
                   big.mark=",", trim=TRUE)
maxLabel <- format(max(testFrame$April.2011),
                   big.mark=",", trim=TRUE)
medianLabel <- format(median(testFrame$April.2011),
                      big.mark=",", trim=TRUE)

#now generate the plot
g <- ggplot(testFrame) +
  aes(x=popChange, y=percentChange,
      size=April.2011, color=April.2011,
      shape=keyState) +
  geom_point() +
  geom_text(data=keyStateDF,
            aes(label=Region, hjust=1, vjust=1))

#add the color scale
g + scale_color_continuous(name="Pop",
                           low = "white", high = "black",
                           breaks = with(testFrame,
                                         c(min(April.2011), median(April.2011),
                                           max(April.2011))),
                           labels = c(minLabel, medianLabel, maxLabel))
```

The output is as follows:



In this visualization it is now easy to see the three states of greatest interest because of their population growth- Georgia, Florida, and Texas. By using **scale_color_continuous( )** with a **black-and-white** coloring scheme, we have effectively put many of the states with low growth into the background. Our labelling is also much less cluttered because we have only labeled the high growth states.

Important R Functions

- **round( ):** Rounds a number (or vector of numbers).
- **hist():** Generates a histogram using R's base graphics.
- **barplot( ):** Generates a bar chart using R's base graphics.
- **data.frame():** Organize variables into a rectangular data structure.
- **filter( ):** Tidyverse function to subset rows of data.
- **format( ):** Format an R object for printing.
- **fromJSON( ):** Extract data from JSON-formatted input.
- **function( ):** Define a function for later reuse.
- **ggplot( ):** Starts the creation of a ggplot, defining the dataframe to be used.
- **group_by( ):** Tidyverse function to organize data into groups.

- **summarize( ):** Tidyverse function to create a data summary.

Functions to Add Layers to a ggplot

- **ggtitle( ):** Adds a title to the plot.
- **geom_histogram( )** Creates a histogram.
- **geom_boxplot( )**: Creates a boxplot.
- **geom_line( ):** Creates a line chart.
- **geom_col( ):** Creates a bar chart.
- **geom_point( ):** Adds points to your plot.
- **geom _text( ):** Adds text to your plot.
- **coord_flip( ):** Rotates the chart by 90 degrees.
- **theme( ):** Refines the visual look of the chart (e.g., text).
- **format( ):** Enables a number to be printed in a pretty, easy-to-read format.
- **scale_color _continuous( ):** Defines a color range for mapping data to a color.